

02 Multimodal Large Model: Voice Control

1. Brief Game Instructions

When the program starts, WonderEcho Pro announces "I am ready" and immediately begins monitoring the surrounding environment for sounds.

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, Hiwonder") to activate WonderEcho Pro. It will respond with "I am here."

You can then control the MentorPi car using voice commands—for example, "Move forward for 2 seconds, then rotate clockwise." The car converts your spoken commands into text via the large model's API and processes them accordingly. After processing, the large model's response is played back aloud, and the corresponding action is executed.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "**20.1 Voice Module Overview and Installation\ 03 Firmware Flashing Method**" for step-by-step instructions.

If you have previously flashed the device with different firmware—such as the version with the wake word “小幻小幻” or “mentorpi”—you must use the corresponding wake word to activate the device.

For example, if the “小幻小幻” firmware was installed, the correct wake word

would be “小幻小幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, HiWonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section "02 Multimodal Model Applications > 01 Obtaining and Configuring Large Model API Keys" to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration Courses\ 01 Network Configuration Instructions”

3. Starting and Stopping the Game

Note:

-
- ① The input commands must strictly observe case sensitivity and spacing.
-

② **The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet.**

1) Start the robot and follow the instructions in “8. Remote Tool Installation Connection → 8.4 Docker Container Introduction and Entry” to connect via the VNC remote desktop software. Then, switch to the ROS2 environment and select the appropriate language version as needed.

2) On the Raspberry Pi desktop, double-click  to open the command-line terminal and enter the ROS2 development environment.

3) Then, input the command to disable the auto-start service:

```
~/stop_ros.sh
```

```
> ~/stop_ros.sh  
zsh: killed ~/stop_ros.sh
```

4) Then, input the command to start the game:

```
ros2 launch large_models llm_control_move.launch.py
```

```
当前环境是ROS2|The current environment is ROS2  
-----  
LIDAR: L019  
CAMERA: ascamera  
MACHINE: MentorPi_Acker  
HOST: /  
MASTER: /  
ROS_DOMAIN_ID: 50  
VERSION: [v1.9.0|2024-10-24]  
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history  
[vocal_detect-7] [INFO] [1742351668.558054472] [vocal_detect]: start
```

5) Once the command line outputs the message below and the device announces “I am ready,” it indicates that the voice device has been initialized. You can now say the wake word, "hello hi wonder."

```
[vocal_detect-7] [INFO] [1742351668.558054472] [vocal_detect]: start
```

6) When the following output appears in the command line, WonderEcho Pro will announce "I am Here," indicating that WonderEcho Pro has been successfully activated. At this point, the system will begin recording the user's command.

- 7) You can freely phrase your commands to control the movement of MentorPi, such as saying, **"Move forward for 2 seconds, then rotate clockwise."**

```
[vocal_detect-7] [INFO] [1747362375.337979087] [vocal_detect]: asr...
[vocal_detect-7] [INFO] [1747362375.911288655] [vocal_detect]: Recording.....
[vocal_detect-7] [INFO] [1747362380.724312320] [vocal_detect]: Done recording
[vocal_detect-7] [INFO] [1747362382.230971298] [vocal_detect]: asr time: 1.51
[vocal_detect-7] [INFO] [1747362382.856225988] [vocal_detect]: publish asr result:Move forward for 2 seconds, then rotate clockwise.
```

- 8) When the command line shows the output below, it indicates that the cloud-based large speech model's speech recognition service has successfully processed the user's command audio. The recognition result will be displayed under "publish_asr_result."

```
[vocal_detect-7] [INFO] [1747362382.856225988] [vocal_detect]: publish asr result:Move forward for 2 seconds, then rotate clockwise.
[vocal_detect-7]
[agent_process-8] [INFO] [1747362382.856828856] [agent_process]: thinking...
```

- 9) When the terminal displays the following output, it means the Raspberry Pi has successfully accessed the cloud-based large language model, processed the user's command, generated a spoken response, and created a corresponding action group based on the command's intent.

At this point, the robot will execute the actions outlined in the "action" sequence.

Note:

The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

```
[llm_control_move-10] [INFO] [1747362385.548193217] [llm_control_move]: {'action': [[0.2, 0.0, 0.0, 2.0], [0.0, 0.0, 1.0, 1.0], [0.0, 0.0, 0.0, 0.0]], 'response': '2 seconds ahead, then a spin!'}
```

- 10) When the following output appears in the terminal, it indicates that the Raspberry Pi has successfully called the cloud-based voice synthesis service. WonderEcho Pro will then play the audio generated from the "response" in step 9.

```
"response": "2 seconds ahead, then a spin!"}
```

- 11) Once this output appears, it means the current round of interaction has concluded. You can then say the wake word again to re-activate WonderEcho Pro and begin a new round of dialogue.

```
[tts_node-9] [INFO] [1747362388.922163920] [tts_node]: [TTS] Playback task complete.  
[vocal_detect-7] [INFO] [1747362388.926243682] [vocal_detect]: enable_wakeup
```

- 12) To exit this feature, simply press “Ctrl+C” in the terminal. If it doesn’t close on the first attempt, you may need to press it multiple times.

4. How It Works

Once the activity starts, you can freely use voice commands to control the MentorPi car to perform actions such as moving forward or backward, and turning left or right—either individually or in sequence. (For the mecanum wheel version, it can also perform lateral movements as well as clockwise and counterclockwise rotations.)

For the rules governing text responses and action design by the large model, please refer to the prompt settings within the program. Detailed instructions can be found in the PROMPT section of

/home/ubuntu/ros2_ws/src/large_models/large_models/function_call.py

The actions available in this mode are predefined within the program library and are limited in number. MentorPi’s movements are controlled by calling functions that correspond to action strings designed by the large model. If a command involves an action that is not predefined, the program will be unable to execute it; however, the large model will still recognize the command and provide a response.

5. Brief Program Analysis

The launch file is saved in:

`/home/ubuntu/ros2_ws/src/large_models/launch/llm_control_move.launch.py`

```
10 def launch_setup(context):
11     mode = LaunchConfiguration('mode', default=1)
12     mode_arg = DeclareLaunchArgument('mode', default_value=mode)
13
14
15     machine_type = EnvironmentVariable('MACHINE_TYPE', default_value='default_machine_type')
16
17     log_machine_type = ExecuteProcess(
18         cmd=['echo', 'MACHINE_TYPE is set to: ', machine_type],
19         shell=True
20     )
21
22     controller_package_path = get_package_share_directory('controller')
23     large_models_package_path = get_package_share_directory('large_models')
24
25     controller_launch = IncludeLaunchDescription(
26         PythonLaunchDescriptionSource(
27             os.path.join(controller_package_path, 'launch/controller.launch.py'))
28     )
29
30     large_models_launch = IncludeLaunchDescription(
31         PythonLaunchDescriptionSource(
32             os.path.join(large_models_package_path, 'launch/start.launch.py')),
33         launch_arguments={ 'mode': mode }.items(),
34     )
35
```

◆ Read the Chassis Type

Send the corresponding prompt to the large model based on the selected underlying platform.

◆ Motion Control

The controller starts the motion control node, which manages the motors, servos, buzzer, and other components on the expansion board.

◆ Large Model Control Node

The start command launches the large model control node, enabling functions such as speech detection, semantic understanding, task execution, and response playback.

5.1 Main Control Program Analysis

The source code is located at:

`/home/ubuntu/ros2_ws/src/large_models/large_models/llm_control_move.py`

```
1 #!/usr/bin/env python3
2 # encoding: utf-8
3 # @Author: Aiden
4 # @Date: 2025/03/06
5 import time
6 import rclpy
7 import threading
8 import json
9 from speech import speech
10 from rclpy.node import Node
11 from geometry_msgs.msg import Twist
12 from std_msgs.msg import String, Bool
13 from std_srvs.srv import Trigger, SetBool, Empty
14
15 from large_models.config import *
16 from large_models.srv import SetModel, SetString, SetInt32
17 from ros_robot_controller_msgs.msg import SetPWMServoState, PWMServoState
18
19 from rclpy.executors import MultiThreadedExecutor
20 from rclpy.callback_groups import ReentrantCallbackGroup
```

◆ Determine whether the input is in Chinese and if the chassis is a mecanum wheel type.

```
22 if os.environ["ASR_LANGUAGE"] == 'Chinese':
23     if os.environ["MACHINE_TYPE"] == 'MentorPi_Mecanum':
```

◆ Library Files Import

The program imports the following modules and libraries to enable functionalities such as ROS2 node creation, message publishing, service calls, threading, and audio playback:

◆ Creates and initializes the ROS2 node

It initializes the ROS2 environment by calling `rclpy.init()` and creates a node instance by inheriting from the `Node` class.

```
101 class LLMControlMove(Node):
102     def __init__(self, name):
103         rclpy.init()
104         super().__init__(name)
```

◆ Initializing Variables

Define the following variables:

action: list of actions

llm_result: string containing the response from the large language model

running: flag indicating the running status

result: parsed result data

```
106     self.action = []
107     self.llm_result = ''
108     self.running = True
109     self.action_finish = False
110     self.play_audio_finish = False
```

◆ Publisher and Subscriber Initialization

- ① tts_text_pub: This publisher publishes text messages to the tts_node/tts_text topic. Sending text to the speech synthesis module: This allows the robot to provide voice feedback to the user.
- ② mecanum_pub: This publisher sends Twist messages to the /controller/cmd_vel topic. The Twist message includes the robot's linear velocities (x, y) and angular velocity (z) to control its movement.
- ③ servo_state_pub: This publisher sends SetPWMServoState messages to the ros_robot_controller/pwm_servo/set_state topic to control the servo motor states.
- ④ llm_result_callback: Subscribes to the agent_process/result topic, which contains commands generated by the Large Language Model (LLM).
- ⑤ play_audio_finish_callback: Subscribes to the tts_node/play_finish topic, which notifies the program when voice playback is complete.


```

112 timer_cb_group = ReentrantCallbackGroup()
113 self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
114 self.create_subscription(String, 'agent_process/result', self.llm_result_
callback, 1)
115 self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_fi
nish_callback, 1, callback_group=timer_cb_group)
116 self.set_model_client = self.create_client(SetModel, 'agent_process/set_m
odel')
117 self.set_model_client.wait_for_service()
118
119 self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wake
up')
120 self.awake_client.wait_for_service()
121 self.set_mode_client = self.create_client(SetInt32, 'vocal_detect/set_mod
e')
122 self.set_mode_client.wait_for_service()
123 self.set_prompt_client = self.create_client(SetString, 'agent_process/set
_prompt')
124 self.set_prompt_client.wait_for_service()
125 self.mecanum_pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)
126 self.servo_state_pub = self.create_publisher(SetPWMServoState, 'ros_robot
_controller/pwm_servo/set_state', 10)

```

◆ Process Action (Process)

Upon receiving a command from the LLM, the script parses the JSON data to identify the action to be performed, such as moving forward or rotating.

```

83     def process(self):
84         while self.running:
85             if self.llm_result:
86                 msg = String()
87                 if 'action' in self.llm_result:
88                     try:
89                         result = json.loads(self.llm_result[self.llm_result.find('{'):self
.llm_result.rfind('}') + 1])
90                     except json.JSONDecodeError as e:
91                         self.get_logger().error(f"JSONDecodeError: {e}")
92                         self.llm_result = ''
93                         continue
94
95                 self.get_logger().info(str(result))
96                 action_list = []
97                 if 'action' in result:
98                     action_list = result['action']
99                     self.get_logger().info(f"Extracted action_list: {action_list}")
00                 if 'response' in result:
01                     response = result['response']
02                     self.get_logger().info(f"Extracted response: {response}")
03                 msg.data = response
04                 self.tts.text_pub.publish(msg)

```